Aalto University
School of Science
Master's Programme in ICT Innovation - EIT Digital Master School

Alberto Xamin

Analysis of Infrastructure as Code for Compliance

Master's Thesis
Espoo, 27 April, 2024

Supervisor: Professor Aura Tuomas, Aalto University
Advisors: Jacopo Bufalino, M.Sc., Aalto University
Nicolas Renaud, M.Sc., Datadog

Aalto University
School of Science
Master's Programme in ICT Innovation - EIT Digital Master School

ABSTRACT OF MASTER'S THESIS

Author:	Alberto Xamin		
Title:	Analysis of Infrastructure as Code for Compliance		
Date:	27 April, 2024	**Pages:**	51
Major:	Autonomous Systems	**Code:**	ELEC3055
Supervisor:	Professor Aura Tuomas		
Advisors:	Jacopo Bufalino, M.Sc., Aalto University Nicolas Renaud, M.Sc., Datadog		

The goal of this project was to create an Infrastructure as Code compliance tool specifically targeting Terraform configuration files. The tool was implemented as a command in an existing open-source tool called *datadog-ci*, which enables integration with continuous integration and continuous delivery pipelines. The tool can scan Terraform plan files for misconfigurations or incompatible resources. To ensure compatibility with the company's internal resource schedule, a resource aggregation algorithm was implemented to merge subordinate resources. The evaluation of the security rules is performed using the Open Policy Agent, which allows the use of a single policy language and model for all products and services. While the current implementation is functional, there are limitations in its scope and future work includes expanding support to more platforms and integrating the scanning logic with a cloud service within the enterprise infrastructure.

Keywords: Infrastructure as Code compliance, Terraform configuration files, Compliance tool development, Continuous integration and continuous delivery (CI/CD) pipelines, Misconfigurations detection, Incompatible resources identification, Resource aggregation algorithm, Security rules evaluation, Open Policy Agent (OPA), Expansion to multiple platforms, Cloud service integration, Enterprise infrastructure integration

Language: English

Aalto-yliopisto
Perustieteiden korkeakoulu
ICT Innovation maisteriohjelma

DIPLOMITYÖN
TIIVISTELMÄ

Tekijä:	Alberto Xamin		
Työn nimi:	Infrastruktuuri koodina-analyysi vaatimustenmukaisuuden varmistamiseksi		
Päiväys:	27. Huhtikuuta 2024	**Sivumäärä:**	51
Pääaine:	Autonomiset järjestelmät	**Koodi:**	ELEC3055
Valvoja:	Professori Aura Tuomas		
Ohjaajat:	Diplomi-insinööri Jacopo Bufalino (Aalto yliopisto) Diplomi-insinööri Nicolas Renaud (Datadog)		
Asiasanat:	Infrastructure as Code compliance, Terraform configuration files, Compliance tool development, Continuous integration and continuous delivery (CI/CD) pipelines, Misconfigurations detection, Incompatible resources identification, Resource aggregation algorithm, Security rules evaluation, Open Policy Agent (OPA), Expansion to multiple platforms, Cloud service integration, Enterprise infrastructure integration		
Kieli:	Englanti		

Aalto-universitetet
Högskolan för teknikvetenskaper
Magisterprogrammet i ICT Innovation

SAMMANDRAG AV DIPLOMARBETET

Utfört av:	Alberto Xamin		
Arbetets namn: Analys av infrastruktur som kod för efterlevnad.			
Datum:	27 April 2024	**Sidantal:**	51
Huvudämne:	Autonoma system	**Kod:**	ELEC3055
Övervakare:	Professor Aura Tuomas		
Handledare:	Diplomingenjör Jacopo Bufalino (Aalto Universitetet) Diplomingenjör Nicolas Renaud (Datadog)		
Nyckelord:	Infrastructure as Code compliance, Terraform configuration files, Compliance tool development, Continuous integration and continuous delivery (CI/CD) pipelines, Misconfigurations detection, Incompatible resources identification, Resource aggregation algorithm, Security rules evaluation, Open Policy Agent (OPA), Expansion to multiple platforms, Cloud service integration, Enterprise infrastructure integration		
Språk:	Engelska		

Acknowledgements

I wish to thank my family, my fellows Tekkaris, especially Riccardo, that supported me during my thesis writing!
Also has to be noted that LLM based tools were used to aid proof reading the text!

Espoo, 27 April, 2024

Alberto Xamin

Abbreviations and Acronyms

API	Application Programming Interface
AWS	Amazon Web Services
BDD	Behavior-Driven Development
CIAA	Confidentiality, Integrity, Availability, and Accountability
CD	Continuous Delivery
CDN	Content Delivery Network
CI	Continuous Integration
CSP	Cloud Service Provider
CSPM	Cloud Security Posture Management
DAG	Directed Acyclic Graph
EEA	European Economic Area
ELB	Elastic Load Balancer
EU	European Union
GCP	Google Cloud Platform
GDPR	General Data Protection Regulation
HIPAA	Health Insurance Portability and Accountability Act
HCL	HashiCorp Configuration Language
IaaS	Infrastructure as a Service
IaC	Infrastructure as Code
JSON	JavaScript Object Notation
NFR	Non-Functional Requirement
OPA	Open Policy Agent
PaaS	Platform as a Service
PCI DSS	Payment Card Industry Data Security Standard
SCM	Source Control Management
SaaS	Software as a Service
SOC 2	Service Organization Control 2
VCS	Version Control System

Contents

Abbreviations and Acronyms **6**

1 Introduction **9**

2 Background **11**
2.1 How infrastructure used to be deployed 11
2.2 Cloud infrastructure 12
2.3 Version control systems 13
2.4 Infrastructure as Code 14
2.5 Terraform 16
2.5.1 Terraform resource dependency graph 17
2.6 The problem of compliance 18
2.6.1 What are the difficulties? 19
2.6.2 How to enforce compliance? 19
2.6.3 Risks of non-compliant infrastructures 20
2.6.4 Strict regulations apply to this field 20
2.7 Misconfiguration and compliance assurance in the cloud CSPM 21
2.7.1 Datadog specific products 21

3 Related Work **23**
3.1 Terraform Compliance 23
3.2 KICS 23
3.3 Regula 24
3.4 Terrascan 24
3.5 Checkov 24

4 Motivation **25**
4.1 State of the art 25
4.1.1 Limitations 26
4.2 Shifting left 26

5 Designing a Compliance Solution for IaC **28**
5.1 Business requirements 28
5.2 How to find misconfigurations 29
5.2.1 Deciding what to scan 29
5.2.1.1 HCL definitions 30
5.2.1.2 Terraform plans 31
5.2.2 Decision 31
5.2.3 Where to run the scan 32
5.2.4 Decision 34

6 Implementation **36**
6.1 Tool architecture 36
6.2 Resource aggregation 36
6.3 OPA and rule structure 39
6.4 GDPR Rule Example 40
6.4.1 Example GDPR Rule for AWS ELB Logging 41
6.4.2 Non-compliant terraform resource example 42
6.5 Reporting non-compliant resources 42
6.6 Limitations 43

7 Evaluation and discussion **45**

8 Conclusion **48**

Chapter 1

Introduction

This thesis is about developing a tool to analyze infrastructure code for compliance with security policies and regulations. The goal of this paper is to present a novel compliance scanning solution specifically designed for Terraform infrastructure-as-code configurations. Maintaining regulatory compliance is crucial for organizations handling sensitive data, as non-compliant systems can lead to data breaches, legal consequences, and loss of customer trust.

The motivation behind this work is to shift compliance checks earlier in the development life-cycle, allowing issues to be identified and resolved before infrastructure is provisioned. Traditional compliance tools only scan deployed cloud resources, missing opportunities to catch misconfigurations proactively. The methods used include a literature survey of existing compliance tooling, analysis of Terraform's architecture, and the implementation of a new compliance scanner integrated with continuous integration pipelines. The main outcomes are:

- An open-source compliance scanner for Terraform plan files
- Techniques for resource dependency analysis and rule evaluation
- Integration with GitHub for compliance feedback during code review

While the current implementation focuses on Terraform, the principles can extend to other infrastructure-as-code frameworks. Future work involves incorporating the scanner into Datadog's Cloud Security Posture Management product.

The rest of the thesis is structured as follows. Chapter 2 discusses the necessary background material for this thesis before providing a thorough

understanding of the topic of misconfiguration and compliance assurance in the cloud. Chapter 3,surveys prior work in this area. In Chapter 4, we go over why this issue requires a fresh approach. Chapter 5 discusses the design choices and Chapter 6 implementation specifics of our suggested solution. Finally, Chapter 8 concludes the thesis.

Chapter 2

Background

This chapter gives the reader the necessary context and concepts which will be required to follow the discussion in the later chapters. The first section describes the evolution of the software infrastructure over the decades. It continues with an overview of legislation and how it applies to software programs and then concludes with a discussion of the technical requirements deriving from the regulation.

2.1 How infrastructure used to be deployed

Around the turn of the millennium, hosting an application was still largely limited to on-premises solutions or renting space in a data center. Web-based software became more popular and internet usage increased as a result of the dot-com boom, but scaling these applications to serve an expanding audience remained challenging. Virtualization technology was just starting to emerge, allowing multiple virtual servers to run on a single physical server, but it had not yet been widely adopted. Cloud computing did not really take off until the second decade of the 2000s when it started to provide a new way to host and scale apps without requiring a sizeable upfront investment in physical infrastructure. Therefore, if you wanted to host an application, you did not have much of a choice; either your company would purchase servers and networking equipment and put it in a dedicated space on the company premises, or it would rent spaces inside a data center to house the equipment. Then, someone would be charged with installing the required software on these physical machines and configuring the different applications and firewalls. However, as a direct correlation with more people suddenly having access to the internet, applications started to be used by more users, and scaling software became a concern.

With physical infrastructure, you have two possibilities for scaling an application; either the company buys a more powerful server or you distribute the load across multiple machines. The latter is cheaper and also has added benefit of connecting the users through the closest server to have faster access to the data. However, a downside of that solution is setting up multiple servers with the same configuration, which is a repetitive task and error prone. Hence, several companies saw the opportunity to release tools to automate this process (e.g. CFEngine, Puppet, Chef, Ansible [4, 11, 19]).

2.2 Cloud infrastructure

In recent years the industry standard of infrastructure deployment has shifted away from bare metal in house, where companies would purchase expensive servers and network equipment to host their applications. The current industry standard is to rely on cloud service providers (e.g,. Amazon Web Services AWS, Google Cloud Platform GCP, Microsoft Azure) as the upfront cost of purchasing the infrastructure was not affordable for most businesses.[26] What Cloud Service Providers offer is the ability to create virtual instances of servers and logical network devices on demand. Moreover, they provide a diverse array of services and features to facilitate infrastructure deployment and management, which can encompass:

- Storage solutions, such as object storage, block storage, and databases
- Content delivery networks (CDNs) for faster content delivery to users
- Load balancing and auto-scaling capabilities to optimize application performance
- Security services, such as firewalls and identity and access management
- Monitoring and logging tools to help with troubleshooting and performance optimization
- Serverless computing options, which allow developers to focus on writing code without worrying about infrastructure management

CSPs also offer a variety of pricing models, including pay-as-you-go, reserved instances, and spot instances, which can help businesses reduce their infrastructure costs depending on usage patterns.

2.3 Version control systems

The development style has evolved over the years, and one important aspect to consider is Version Control Systems (VSC), also known as source control. VSC practices track and manage changes to software source code. When it comes to Version Control Systems or Source Control Management (SCM) tools, there is more than just the binary choice between centralized or distributed systems.

Git, Subversion (SVN), Mercurial, Perforce, and Team Foundation Server are some of the Version Control Systems, each with its own set of advantages and disadvantages. The choice of which system to use depends on various factors, including the size of the team, the complexity of the project, and the specific development workflow in place. So, besides the centralized or distributed forms of VSC, there is a spectrum of options to consider when choosing the right tool for managing source code. However the most prominent now is git, some large organizations are occasionally relying on in-grown alternatives. [27]

Software engineers continuously create new source code and make alterations to existing code, especially when collaborating within teams. Project, application, or software code is typically organized within a hierarchical folder structure. Developers often make changes in multiple parts of this structure, with one developer focused on implementing a new feature while another concentrates on addressing a separate bug.

Version Control is crucial for managing infrastructure because it allows for traceability by keeping a record of the changes made, including who made them and why. It also enables the ability to rollback changes that cause functionality issues, allowing teams to revert to a previously working state quickly.
If all components of a system are tracked by the version control system, then tracing and fixing complex problems becomes manageable.
Everyone can see when and what changes are made, and if an incident or outage happens, people are aware of the recent changes that might have caused it.
Furthermore, the developers contributing on the project can automatically trigger commands when changes are committed; this helps with continuous automated deployments. Automated deployments refer to the process of deploying changes to a system in an automated manner, without requiring

manual intervention. Integration of version control systems with deployment tools enables the automatic triggering of deployments upon the committing of changes to the repository. This practice streamlines the deployment process and mitigates the risk of errors and inconsistencies that may arise from manual deployments. Moreover, automated deployments facilitate the acceleration and intensification of deployment cycles, resulting in faster and more agile development.

2.4 Infrastructure as Code

Infrastructure as Code is defined as the use of software development methods to infrastructure automation. For provisioning and modifying system configurations, the emphasis is on repeatable and consistent operations. The definition of the changes is usually made on text files that are rolled out through automatic processes that have built-in validation.

Infrastructure entities, such as physical computers and network devices, have been supplanted by virtual counterparts through the power of virtualization and cloud technology. These resources can be created, maintained, and removed programmatically. Cloud Service Providers (CSPs) have introduced interfaces to significant configuration management systems, enabling the description of infrastructure resource states. Additionally, major CSPs have introduced their Infrastructure as Code (IaC) languages tailored for deployment on their respective platforms: Amazon Web Services introduced CloudFormation in 2011, Google launched Cloud Deployment Manager in 2014, and Microsoft introduced Azure Resource Manager in 2015.

Imperative and declarative languages are two distinct paradigms utilized in software development and infrastructure management. Imperative languages, such as conventional software programming languages, prescribe a sequence of steps to achieve a desired outcome. The programmer explicitly specifies how the program should execute, step by step, and is responsible for managing state and control flow. Conversely, declarative languages, such as IaC, concentrate on describing the intended outcome rather than the specific steps to achieve it. The user specifies the desired configuration or state of a system, and the tool or platform takes care of the implementation details.

Declarative languages provide a more abstract and declarative approach, allowing users to define infrastructure configurations using high-level abstractions that are independent of the underlying platform or infrastructure. This

approach simplifies the management of complex infrastructure and enables users to manage infrastructure as code, which can be versioned, tested, and automated. Imperative languages, in contrast, require users to manage the low-level details of infrastructure management, which can be time-consuming and error-prone. To manage infrastructure, individuals can utilize the interfaces offered by cloud providers or third-party solutions equipped with graphical interfaces for interactive infrastructure management. However, when the aim is to achieve infrastructure that is repeatable, reusable, testable, and self-documenting, Infrastructure as Code becomes imperative. Repeatable, reusable, and testable infrastructure are essential attributes crucial for efficient and effective infrastructure management.

Reusable infrastructure enables users to leverage existing infrastructure configurations to build new infrastructure, saving time and effort and ensuring consistency and standardization across infrastructure deployments. Testable infrastructure allows for the testing of infrastructure configurations to ensure that they meet the desired requirements and specifications, thus facilitating the identification and resolution of issues before they impact production environments. Testing also helps to ensure that infrastructure configurations are reliable, performing, and secure.
The prioritization of repeatable, reusable, and testable infrastructure enables organizations to achieve greater efficiency, consistency, and reliability in their infrastructure management practices. This approach can lead to cost reduction, improved agility, and enhanced overall quality of infrastructure deployments. Consequently, organizations can benefit from increased productivity, reduced risk, and improved customer satisfaction.

One might think that a simple script that can run on a new machine to install and configure it would count as Infrastructure as Code; however more often than not, these scripts require manual steps, such as needing a person to copy the script to the new machine and to execute it. Scripts with these manual attention requirements do not allow for a truly automated infrastructure and block development teams from more important non-repetitive tasks. A number of problems that may occur when managing infrastructure directly can be avoided with the use of IaC. Inconsistent configuration is one such problem, which can cause differences between environments, mistakes, and security vulnerabilities. Lack of documentation is another problem, which can make it challenging to comprehend how infrastructure is configured and managed. Human error, such as forgetting to set a resource or incorrectly configuring a resource, can result in security vulnerabilities when the configuration involves manual steps. Finally, because it takes a lot of time and effort

to set each individual resource, manual configuration can be challenging to scale. By utilizing IaC, infrastructure can be handled in a standardized, repeatable, and automated manner, lowering the possibility of mistakes and security flaws. IaC can also enhance documentation and scalability, making infrastructure management simpler as infrastructure expands and changes over time. For security reasons, this can be especially crucial as it can help ensure that infrastructure is configured correctly and consistently, lowering the risk of security vulnerabilities and ensuring that resources are properly secured.[9]

In this context of a software development process a *blocker*[6, 22, 23] refers to any issue or obstacle that impedes progress or hinders the smooth execution of tasks. To free teams from such obstacles and enable the execution of infrastructure configuration scripts without the need for constant supervision, the following conditions should be met:

- Idempotent. Running the script multiple times should always lead to the same configuration state [5, 10, 14].
- Failing visibly. If a task fails to complete, it should be visible to the people responsible to act accordingly.

2.5 Terraform

Terraform, a HashiCorp software product, was launched in 2014. Terraform interprets declarative infrastructure specifications defined in the HashiCorp Configuration Language (HCL) and communicates with various CSPs' Application Programming Interfaces (APIs) to furnish the architectures described. Terraform has the advantage of introducing a syntactically uniform language to describe and materialize cloud infrastructures; however, the code interface of the modules used to communicate with different CSPs is still provider-specific, and vendor lock-in remains a major concern when using IaC to manage cloud infrastructure.[3, 12, 18, 20]

The main concepts of Terraform can be broken down into these key elements.

- **Configuration**: This is described through the high-level HCL language and represents the desired *end-state* of cloud or on-premise infrastructure.
- **State**: It stores the state of the managed infrastructure and is used to map real-world resources to the definitions in the configuration files.

- **Planning Operation**: This operation compares and computes the differences between the state and the configuration.
- **Apply Operation**: This is an action that resolves the differences and updates the real-world resources.

2.5.1 Terraform resource dependency graph

In the context of Terraform, several key concepts are important for the creation of the resource dependency graphs. *State* represents the current state of managed resources, encompassing their attributes and dependencies, and is stored in a state file. The term "available" pertains to the accessibility of this state information, indicating whether Terraform can reference it for change management. *Provisioners* are configuration blocks used to execute actions on resources post-creation or update, facilitating further configuration. In the context of a resource dependency graph, *edges* denote dependencies between nodes, reflecting the relationships between resources. An *orphan* resource is one present in the state but absent in the configuration, necessitating a separate treatment to avoid unintended deletions. *Interpolation* enables dynamic value insertion within configuration files, offering flexibility in referencing variables, resource attributes, or other expressions. Lastly, *diff* involves comparing the current state with the desired state to determine necessary actions, such as creating, updating, or destroying resources, in Terraform's infrastructure management process. Understanding these concepts is essential for comprehending Terraform's orchestration of infrastructure as code. Terraform operates on a resource dependency graph which is composed of three different types of nodes: Resource nodes that are based on the resources defined in the configuration files; Provider configuration nodes that represent the time required to fully configure a provider; and Resource Meta-Nodes that represent a group of resources; the scope of nodes is to make the graph easier to operate.

The creation of the nodes happens by the following procedure:

1. Resource nodes are added based on the configuration file. If a state is available, the related metadata is loaded into the resources.
2. Resources are mapped to their provisioners, if they have any defined. This must be done after all resource nodes are created so resources with the same provisioner type can share the provisioner implementation.
3. Edges are created based on the *depend_on* meta-parameter.

4. If a resource is present in the state but not in the configuration, the resource is added to the graph as orphans.
5. Resources are mapped to the providers, this adds provider configuration nodes and an edge that connects the resources with their providers.
6. Value interpolations are parsed and turned into dependencies.
7. A root node is added and points to all the resources.
8. If a diff or state is present, all the resources are traversed to find the ones that will be destroyed.
9. Check that the graph is a Directed Acyclic Graph (DAG).

To plan the resources and apply it, the graph needs to be traversed though a depth-first traversal. Nodes at the same level are run in parallel with a semaphore limiting the processing of the resources for which the dependencies have not been satisfied yet. However as some cloud providers implement a rate limiting on their APIs, the number of parallel executions is reduced, as for the AWS provider.

2.6 The problem of compliance

As discussed in earlier sections, the use of cloud service providers has become increasingly prevalent. This has led to a reduction in the barriers to accessing their scalability tools and complex features. However, this ease of use comes with a caveat as it raises concerns about how to handle important matters such as compliance, security, and privacy.

These new challenges have arisen as a result of regulations introduced by governments, aimed at ensuring correct handling of user data.

Regulations are sets of policies that govern the use of sensitive user data. The main goal of these regulations is to protect consumer privacy and security by requiring attributes that include confidentiality, integrity, availability, and accountability (CIAA). The enforcement of the rules that implement the policies established in the regulations is what compliance entails.

For a significant number of software systems, legal compliance may become the most important Non-Functional Requirement (NFR). Government and state rules must be followed, and industry regulations, typically a customer

requirement, might also become a contractual obligation. Although regulations differ from country to country, many adopt similar underlying policies tailored to their own needs. [25].

2.6.1 What are the difficulties?

Regulations, being legal documents, are often characterized by verbosity, length, ambiguity, redundancy, and occasional contradictions, making them difficult to interpret. Despite this complexity, both service providers and consumers are expected to achieve full compliance and often must adhere to multiple regulations simultaneously. Achieving full compliance with a regulation often goes beyond simply avoiding criminal activities. It typically involves adhering to a wide range of legal and operational requirements related to data handling, privacy, security, and more.
Considering the variations in regulations across different countries, the effort required to comply with diverse regulatory regimes worldwide is substantially increased. As there could be contradicting requirements, such as privacy regulations in one country and data retention or lawful access in another.
Furthermore, overlapping regulatory frameworks [7] exacerbate the challenge of implementing individual regulations, leading to elevated maintenance costs. This complexity makes translating legal requirements into code an exceptionally difficult task.

2.6.2 How to enforce compliance?

Cloud Security Posture Management (CSPM) is a software solution that detects security issues in cloud infrastructure and inspects cloud-hosted software, such as Software as a Service (SaaS), Platform as a Service (PaaS), Infrastructure as a Service (IaaS), Containers, and Serverless code.

CSPM is a fully automated system that runs in the background, evaluating the cloud for compliance risks and configuration vulnerabilities, eliminating the need for security teams to manually verify their clouds for security threats.

Most CSPM solutions can scan multi-cloud infrastructures and provide a comprehensive picture of security across all cloud services. This capability is crucial as many businesses employ multiple cloud services, increasing the risk of misconfiguration and making manual management more challenging.

Importantly, CSPM products often implement up-to-date frameworks (ex-

amples in Section 2.7.1) that have already mapped regulations into code, simplifying the process of translating legal requirements into actionable security measures. This feature significantly eases the burden of achieving compliance with complex and diverse regulatory regimes across various countries.

2.6.3 Risks of non-compliant infrastructures

A non-compliant resource is characterized by misconfigurations that potentially expose sensitive information to the public or render it susceptible to attacks. For instance, numerous data breaches could have been averted if appropriate access control mechanisms had been implemented on the relevant S3 buckets. A notable example occurred in June 2017 when the personal data of 198 million American voters was inadvertently leaked due to being stored on an inadequately secured Amazon server[13, 17, 21].
The data breach was discovered by cybersecurity researcher Chris Vickery, who found the information stored on an unsecured Amazon S3 server. The server belonged to Deep Root Analytics, a data analytics firm contracted by the Republican National Committee (RNC).
The exposed data included sensitive information such as names, addresses, dates of birth, phone numbers, and voter registration details. Additionally, the data contained political profiling information, such as voters' political affiliations, voting histories, and opinions on various political issues.
The primary cause of the breach was a misconfigured security setting on the Amazon S3 server, which allowed public access to the stored data. This incident highlighted the importance of proper security configurations and access control measures in cloud storage environments to prevent unauthorized access and potential data breaches.

2.6.4 Strict regulations apply to this field

The General Data Protection Regulation (GDPR) was introduced into European legislation with the goal to strengthen data protection and privacy for individuals in the European Union. According to the GDPR website, the penalties imposed by the regulation are intended to make noncompliance a financially significant mistake for companies of all sizes.
Non-compliant organizations may encounter severe financial penalties under GDPR. The fines are structured in a two-tier scheme, with the most serious infringements carrying fines of up to 20 million € or 4% of the company's worldwide annual turnover, whichever is greater. Lesser infractions can result in fines of up to 10 million € or 2% of the company's worldwide annual turnover, depending on which is higher.

These substantial fines emphasize the importance of GDPR compliance for businesses operating within or dealing with individuals in the EU and EEA. Companies must ensure that they have appropriate data protection measures, policies, and procedures in place to avoid the risk of non-compliance and the potential financial repercussions associated with GDPR violations.

2.7 Misconfiguration and compliance assurance in the cloud CSPM

Misconfigurations in cloud infrastructure can lead to vulnerabilities and security breaches, and ensuring compliance with regulatory standards is important to protect sensitive data and prevent legal consequences. There are various tools and approaches available to address misconfigurations and ensure compliance in the cloud, including Cloud Security Posture Management (CSPM) solutions, Infrastructure as Code (IaC) scanning tools, and manual reviews. These approaches may be used in combination to provide comprehensive coverage and ensure the security and compliance of cloud infrastructure.

Tools for misconfiguration and compliance assurance in the cloud typically work by evaluating a set of rules against the resources defined in a cloud infrastructure configuration, such as those defined in an Infrastructure as Code (IaC) project. These rules can be designed to check for compliance with industry standards or company policies, and may include checks for proper configuration of security settings, resource permissions, and other important factors. The tools may be run on a local machine or on a remote server, and may be integrated into a continuous integration/continuous deployment (CI/CD) pipeline to automatically catch misconfigurations before they are deployed to production. The results of the evaluation are typically reported to the user, highlighting any resources that are not compliant with the defined rules and providing guidance on how to fix any issues that are found.

2.7.1 Datadog specific products

Datadog offers several security and compliance products that help organizations ensure that their cloud infrastructure and applications meet various industry standards and regulations. These include:

- Critical Security Controls (CIS Controls)

Helps organizations implement the Center for Internet Security (CIS) 20 critical security controls, which are a set of best practices for cybersecurity[1].

- Payment Card Industry Data Security Standard (PCI DSS)
  Helps organizations meet the security standards required for handling payment card data[2, 8, 16].

- Systems and Organization Controls 2 (SOC 2)
  Helps organizations demonstrate that they have adequate controls in place to protect the security, confidentiality, and availability of customer data.

- Health Insurance Portability and Accountability Act (HIPAA)
  Helps organizations meet the American security standards required for handling healthcare data[8, 15].

- General Data Protection Regulation (GDPR)
  Helps organizations meet the data protection requirements set by the European Union's GDPR[24].

Overall, these products work by providing a set of tools and services that help organizations identify and fix misconfigurations in their cloud infrastructure and applications, and ensure that they meet the necessary compliance standards. This can include things like monitoring for security vulnerabilities, detecting suspicious activity, and providing guidance on how to fix problems.

Chapter 3

Related Work

This chapter explores existing research and tools in the area of compliance and security checks for Infrastructure as Code (IaC). The term *rules* is used here to refer to security and compliance standards, and *scanning* implies the evaluation of these rules against the resources defined in an IaC project. The primary platform of interest in these studies is Terraform.

3.1 Terraform Compliance

Terraform Compliance is a focused tool used for *scanning* Terraform resources to ensure they adhere to defined security and compliance *rules*. A key aspect of its functionality is the implementation of negative testing. Negative testing, in this context, involves intentionally providing incorrect, invalid, or unexpected input or conditions to the Terraform configurations to verify that they do not pass the compliance checks when they are not supposed to. This approach is crucial in identifying how the system behaves under failure or error conditions, ensuring that only secure and compliant resources are deployed.
In the scanning process, Terraform Compliance uses Terraform Plan Files, and *rules* are defined in *radish* language, adhering to Behavior-Driven Development (BDD) syntax. Users are responsible for creating their own *rules*, requiring knowledge of relevant security risks and practices.

3.2 KICS

Keeping Infrastructure as Code Secure (KICS) is a tool for static code analysis in IaC environments. It supports various IaC solutions, including OpenAPI. KICS uses the *Rego* language for its rule definitions and offers an open-

source set of *rules*. The tool provides scan results in JSON and HTML formats.
KICS is designed to identify security and compliance issues in IaC files through its diverse rule set and support for multiple IaC platforms.

3.3 Regula

Regula is an open-source command-line tool for assessing IaC files for security and compliance violations. It supports various cloud environments and IaC systems. Regula allows for the use of both community-contributed and custom *rules* in the *Rego* language. The tool outputs scan results in JSON. Regula's role is to detect potential security and compliance issues in IaC files, emphasizing early identification and resolution.

3.4 Terrascan

Terrascan analyzes IaC files and is compatible with a range of IaC tools. It includes a pre-defined set of policies and supports the creation of custom *rules* using the *Rego* language. Terrascan can operate in a *server* mode, providing an HTTP API server for programmatic queries.
Terrascan's features, such as its support for multiple IaC platforms and server mode, facilitate the security and compliance evaluation of infrastructure definitions.

3.5 Checkov

Checkov, a tool developed by Bridgecrew, supports multiple IaC platforms. It features a GitHub app for monitoring repositories and generating compliance reports. Checkov integrates with communication platforms like Slack for alerts.
Checkov provides guidelines for addressing non-compliant resources and includes an auto-remediation feature for Terraform, which creates corrective pull requests in the monitored repository. This feature is aimed at streamlining the compliance management process.

Chapter 4

Motivation

In this chapter, we outline the limitations of current compliance technologies implemented by Datadog and proposed an alternative approach to address these issues. For the purposes of this research, we have chosen to focus on Terraform as the target Infrastructure as Code tool. Our goal is to develop a solution that can effectively identify and prevent misconfigurations and vulnerabilities in infrastructure as code, while also minimizing the impact on development speed and efficiency.

4.1 State of the art

Datadog's current solution involves scanning deployed infrastructure, both in custom data centers and on public cloud providers. For custom data centers, an agent program called the Datadog Agent must be installed on the hosting computer to gather information about the operating system and running services and processes. On public cloud providers, the scan begins with a *crawler* process on Datadog's servers, which requires access key credentials to retrieve resource information from the cloud provider's API. Relevant information gathered during the scan includes configuration parameters and resource interconnections. These API calls are made at regular intervals, typically in the range of minutes. Additionally, information can be transmitted from the customer's data center to Datadog's servers via the installed agent process. In this scenario, data originating from on-premises resources is transmitted to Datadog according to the agent's configuration, as determined by the user. A diagram summarizing the data flowis shown in Fig. 4.1.

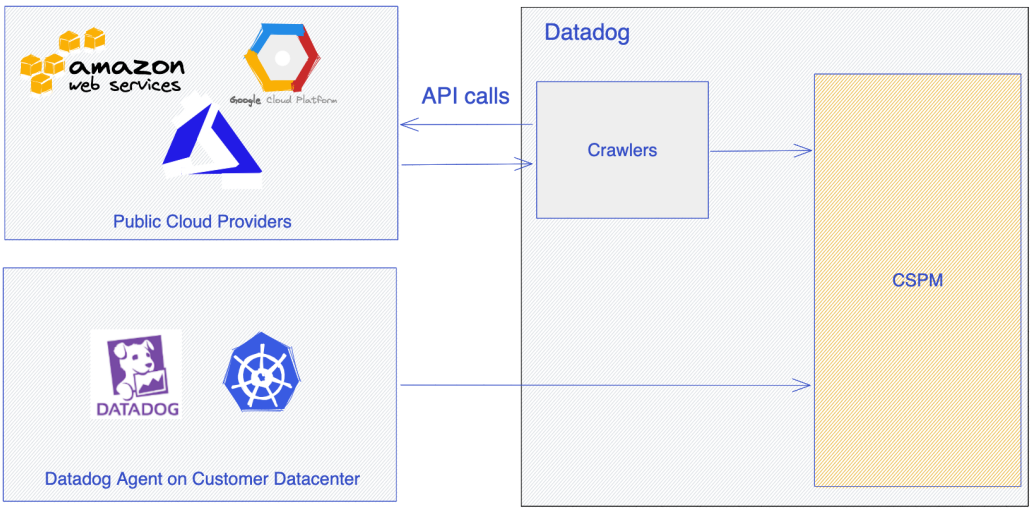

Figure 4.1: High level overview of the current solution

4.1.1 Limitations

There are several limitations. One significant limitation is that it only scans deployed infrastructure, meaning that any misconfigurations that have already been deployed are public and could potentially expose data to attacks. This means that any issue with the infrastructure is not identified and addressed until after those resources have been made available public by a deployment to production, which could potentially have serious consequences.

4.2 Shifting left

Traditionally, many software teams followed a waterfall model, where testing was only performed at the end of the development cycle. However, as the need for faster development has grown, it is increasingly necessary to integrate testing into earlier phases of the cycle rather than leaving it until the final stages. This shift allows for the identification and resolution of issues earlier in the process, helping to speed up overall development and improve the quality of the final product.

As previously stated, numerous companies utilizing Cloud Service Providers (CSPs) rely on Infrastructure as Code (IaC) tools to define their infrastructure. A recommended practice is to incorporate a checking logic for misconfigurations (defects) based on these definitions before deploying the infrastructure, which would prevent the creation of non-compliant and inse-

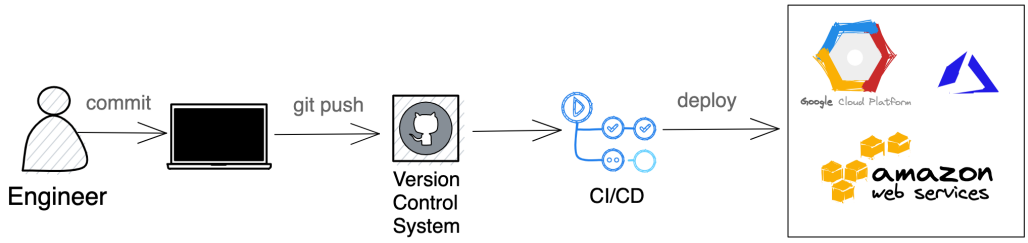

Figure 4.2: Example development pipeline

cure resources. This proactive approach guarantees the deployment of only compliant and secure resources, ultimately enhancing the overall security and compliance of the infrastructure.

One way to improve the security and compliance of infrastructure as code is to integrate scanning at various points in the development pipeline (as illustrated in Fig. 4.2). By implementing checks at different stages of the process, we can reduce the risk of vulnerable changes being committed or merged into version control systems, and prevent the deployment of infrastructure that contains vulnerabilities or non-compliant resources. This approach helps to ensure that only secure and compliant infrastructure is deployed, improving the overall security and compliance of the system.

This scanning approach is conducted with only partial information about the system, and a balance needs to be achieved between the sensitivity of the rules and the scan's coverage to ensure its effectiveness without hindering development. Furthermore, this method is contingent on good practices for managing Infrastructure as Code. Examples of good practices for managing Infrastructure as Code include version control, testing, and peer review. Without proper management, the scan may not identify all potential issues or misconfigurations.

However, even with good management practices in place, some issues may not be identified by the scanning approach. For instance, the scan may not detect issues that arise from interactions between different components of the infrastructure or from changes made outside of the IaC configuration. As such, it is important to supplement the scanning approach with other security measures, such as monitoring and incident response procedures.

Chapter 5

Designing a Compliance Solution for IaC

In this chapter, we begin by discussing the business requirements for an effective compliance solution. We then describe the decision-making process that led to the implementation of a new compliance tool. This includes a review of the options available and an analysis of the potential benefits and drawbacks of each option.

5.1 Business requirements

As a company, Datadog aims to establish a strong presence in the security market. One way it is seeking to achieve this is by expanding its range of products and services, including its existing Cloud Security Posture Management solution. By offering a tool that interfaces with customers' existing and new infrastructure as code, and provides a range of compliance and security checks, Datadog hopes to make its product offerings more competitive in the market and attract more customers. This expansion of its product line will allow Datadog to broaden its reach and increase its market share in the security industry. In order to establish a strong presence in the security market, Datadog has identified several key features that its Cloud Security Posture Management solution must have. Firstly, the company recognizes the importance of Infrastructure as Code (IaC) in modern cloud environments. As such, the solution must have an IaC checker that can scan and evaluate infrastructure-as-code configurations. This feature will enable customers to identify and fix security and compliance issues in their infrastructure before it is deployed, reducing the risk of security breaches and non-compliance. Secondly, it understands that the solution must be able to evaluate rules on

a server provided by Datadog, rather than on the client-side, to ensure that the evaluation process is secure and reliable. By performing rule evaluation on a server, Datadog can ensure that the results are consistent and that customers can trust the output of the solution, as will be explained in section 5.2.3 as long-term preference. Additionally, offering pre-commit hooks allow customers to integrate the IaC checker into their existing development workflows. Pre-commit hooks are scripts that run automatically before a commit is made in a version control system like Git. They check the code for errors or issues, ensuring that only high-quality code is committed. For Datadog's Cloud Security Posture Management solution, integrating pre-commit hooks means enabling early detection of security and compliance problems in Infrastructure as Code (IaC). This helps prevent vulnerabilities from entering the codebase, aligning with Datadog's goal of enhancing security in cloud environments.

By incorporating these "must-have" features into its Cloud Security Posture Management solution, Datadog hopes to provide a more comprehensive and competitive tool for customers to manage their cloud security and compliance needs. This expansion of its product line will allow Datadog to broaden its reach and increase its market share in the security industry.

5.2 How to find misconfigurations

Misconfigurations are a common source of vulnerabilities on public cloud providers, and it is important to identify and address them before they affect public-facing products. In this section, we will explore several options for detecting and preventing misconfigurations, and outline the design decisions that were made to arrive at the proposed solution. Our goal is to provide a clear and comprehensive overview of the considerations that went into the design of this tool, and how it aims to address the problem of misconfigurations in infrastructure as code.

5.2.1 Deciding what to scan

To ensure compatibility with multiple Infrastructure as Code software solutions, an ideal scanning tool would need to support various platforms. However, in order to obtain early feedback from users and establish a strong foundation for the product, it is necessary to initially focus on supporting a single platform. For the reasons explained below, we have chosen to prioritize Terraform as the initial platform to support.

This decision was made for several reasons. Firstly, Terraform is a popular choice among users due to its ability to support multi-cloud infrastructure, making it a cloud agnostic tool. Additionally, Terraform has a large user base and an active community, which can provide valuable feedback and support during the development process.

While Terraform primarily focuses on infrastructure provisioning and configuration, it's worth noting that you can perform scans on the deployed infrastructure. However, this may not be a common or practical approach, as Terraform is typically used for creating and modifying infrastructure as code. Nevertheless, in some scenarios, post-deployment scans can be conducted to assess the security and compliance of the deployed infrastructure. However, it is important to note that the ability to output the result of a planning operation was only added in Terraform 0.12, meaning that earlier versions of Terraform would not be supported. Despite this limitation, focusing on Terraform as the initial platform will allow us to create a strong foundation for the product and expand support to other platforms in the future.

5.2.1.1 HCL definitions

Terraform projects are configured using HashiCorp Configuration Language (HCL). As the name suggests, HCL is a configuration language developed by HashiCorp and is used with a suite of HashiCorp-owned cloud infrastructure automation tools, including Terraform. HCL is a language that allows developers to define and configure their infrastructure in a clear and concise manner, making it easy to understand and modify as needed.

One advantage of supporting HCL is that we can scan the source file directly, which makes it easier to identify the source of any misconfigurations in the code. Additionally, we can avoid the need to authenticate with cloud providers or run planning operations, which would reduce scanning time. However, parsing and evaluating HCL files can be more complex due to the use of value interpolation, which allows users to write expressions that are evaluated at run-time. As a result, it may be more challenging to accurately determine the values of these expressions. Moreover, it is important to note that HCL configuration files are designed to contain only the minimum set of parameters needed to create infrastructure, so they may not provide a complete picture of all resources and configurations.
To illustrate the concept of run-time evaluation with value interpolation, let's consider a simple example snippet. HCL is widely used in infrastructure

as code (IaC) for defining cloud infrastructure in a structured and human-readable format. Value interpolation allows dynamic computation of values at the time the configuration is applied, enhancing flexibility and reusability.

```
variable "instance_type" {
  description = "The type of EC2 instance to create."
  default     = "t2.micro"
}

resource "aws_instance" "example" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = var.instance_type  # Value interpolation

  tags = {
    Name = "ExampleInstance-${var.instance_type}"
  }
}
```

Example of HCL with value interpolation.

5.2.1.2 Terraform plans

Scanning the output of a plan action provides a more accurate representation of the actual infrastructure that will be deployed. The output can also be exported to a machine-readable format (JSON), which makes it easier to process and analyze. However, this approach has the disadvantage of requiring handling of sensitive fields, such as secrets that could be included in the output. In a Terraform plan, we can identify which fields are sensitive as they are reported in a separate object. Additionally, it may be more difficult to trace the origin of non-compliant properties in the source code, which could make it challenging to generate automatic fixes in the future. It is also worth noting that running a planning operation requires authentication with cloud providers and can take a significant amount of time (up to tens of seconds, depending on the number of resources in the state) to complete.

5.2.2 Decision

We have decided to initially focus on supporting files generated by Terraform plans. This approach allows us to quickly gain feedback from users while laying the foundation for future expansion. While this approach requires a minimum version of Terraform 0.12, we believe this will not be a significant issue as most Terraform users are already using at least this version or newer,

based on our consultations with HashiCorp. By focusing on Terraform plans, we can provide a more accurate representation of the infrastructure that will be deployed and leverage the machine-readable format (JSON) for easier processing and analysis as will be explained in section 5.2.3. Additionally, this approach allows us to identify sensitive fields and potentially generate automatic fixes in the future. However, we recognize the importance of supporting multiple platforms and will develop an interoperability schema to enable support for additional Infrastructure as Code solutions in the future. This decision reflects a balance between the benefits and drawbacks of each approach and our desire to deliver business value as soon as possible while also building a scalable and flexible solution.

5.2.3 Where to run the scan

Another important decision that needs to be made is where to run the scan. The logical options are either on the local machine where the plan is generated or on a remote machine, such as a server in the Datadog cloud. There are pros and cons to each approach, and the best option will depend on the specific needs and constraints of the project. For example, running the scan on the local machine may be faster and more convenient, but a remote machine could offer more processing power and resources, as well as the ability to leverage any existing security and compliance measures in place.

Scan on a different machine

Performing the scan on a remote machine would involve sending the files to a server for processing and returning the results to the local machine that initiated the scan. From a business perspective, integrating this feature into the existing open source repository, *datadog-ci*, could be beneficial as it already contains tools for integrating Datadog with customer CI pipelines. The current design of the repository, which primarily consists of wrappers for API endpoints, would also align well with this new feature.

Running the scan on a remote machine would also allow us to leverage the existing knowledge of deployed infrastructure and perform checks based on a more complete picture of the system, rather than just the information managed by Infrastructure as Code tools. In addition, any issues with the scanning logic or rules can be quickly addressed without requiring users to update their software. Furthermore, running the scan on company servers would allow us to protect our intellectual property related to the rules.

In addition to the benefits mentioned, running the scan on a remote machine would also provide a consistent and standardized environment for running scans, which can help ensure consistent and reliable results. This is because the cloud environment can be configured to meet specific requirements, such as specific versions of dependencies or libraries, and can be easily updated or maintained as needed. Furthermore, running the scan on a remote machine would allow Datadog to leverage the existing knowledge of deployed infrastructure and perform checks based on a more complete picture of the system, rather than just the information managed by Infrastructure as Code tools. Any issues with the scanning logic or rules can also be quickly addressed without requiring users to update their software, as the tool can be updated on the server-side.

However, there are some security considerations to take into account for both Datadog and the customer. On the Datadog side, we would need to ensure that custom rules do not waste resources or access unauthorized data. As the rule is arbitrary code that would run on the server, if a rule is not optimized for performance, it could consume a significant amount of resources when processing large Terraform plans. This could impact the overall performance of the scanning tool and potentially cause delays or failures that could also impact other customers of the tool. On the customer side, it is important to properly handle any secrets that may be included in the plan file. It is also worth noting that implementing this approach may introduce some latency due to network communication, and it would require an internet connection to run the tool as a pre-commit hook. Additionally, processing large Terraform plans (which can be up to 50kB) may put strain on the current message broker-based infrastructure, which is not designed for large payloads.

Scan on the same machine

Running the scan for misconfigured resources on the same machine that is running the Terraform plan has several advantages. For example, it would be easier to allow customers to add their own custom rules and there would be less need for sanitization or ensuring that the rules do not do anything malicious. It would also result in a faster execution and a better user experience if customers use pre-commit hooks. Additionally, this approach may be more forgiving when checking values in sensitive fields, although some sensitive values may already be crawled by Datadog services.

However, there are some drawbacks to this approach as well. For exam-

ple, the detection rules would need to be distributed to customers in order to be scanned on their local machines, which could be a challenge for a commercial product. Additionally, since this approach would only have access to data exposed by Terraform, it would be difficult to provide a comprehensive view of a user's cloud infrastructure. An internet connection would still be required for the Terraform plan operation, and it would be difficult to ensure that all customers are using the same version of the command line interface. In order to provide value for customers and justify a paid product, we would need to carefully consider these limitations and find ways to overcome them.

5.2.4 Decision

To assist with the decision-making process, we decided to examine how competitors and existing solutions approach the execution of the scan.

Product	**Local**	**Remote**	**Terraform Support**
Regula	yes	only metrics	yes
Checkov	yes	yes	yes
Hashicorp Sentinel	yes	yes	yes
Terrascan	yes	self hosted	yes
KICS	yes		yes
Terraform Compliance	yes		yes

Table 5.1: Overview of features available in open source tools.

After reviewing the options available on the market, we found that all of the tools offer a local scanning feature, with some also offering more advanced scans in the backend (such as Checkov from Bridgecrew and Sentinel from HashiCorp). Regula, on the other hand, takes a different approach and only reports certain metrics to a backend.

Although running the scan on a separate machine within the Datadog infrastructure would likely be the most effective approach, it would have taken significantly more time to implement than was available for this project. As a result, we decided to run the scan locally. While this may have some limitations, the architecture we have implemented will make it possible to transition to a remote scanning approach in the future. This will allow us to build on the foundation we have laid and continue to improve the solution over time. For a robust, enduring solution, the most accurate approach involves employing a Datadog cloud server to access and analyze customers' configuration data. This data can then be used for research and the devel-

opment of new rules, subject to the customers' consent, aimed at enhancing infrastructure security.

Chapter 6

Implementation

As outlined in the previous chapter, the goal of this chapter is to detail the implementation phase of the compliance tool that was developed during the internship.

6.1 Tool architecture

As mentioned in Chapter 5, the functionality of the tool was implemented as a new command for an existing tool called *datadog-ci*, which allows users to execute commands with Datadog within their CI/CD pipelines.
The architecture of the tool is shown in Figure 6.1. The tool takes a Terraform plan file and parameters as input. The parameters allow users to fine-tune the search for misconfigured resources. The plan file, a JSON file, is parsed to extract the information needed to generate a resource graph. This graph is then mapped to the internal meta schema (described in Section 6.2) for resources on which the rules will be evaluated. The list of resources and the collection of rules are then loaded into the evaluator, which produces a list of non-compliant resources. A quick filter is applied to this list based on the parameters specified as input (e.g. to filter only changed resources). If any misconfigurations are found, they are presented to the user with a failure exit code, which can block CI/CD pipelines.

6.2 Resource aggregation

One of the desired outcomes of the tool is to apply the same set of rules to both resources parsed using the new tool and those scanned by the crawler. However, this is not straightforward because the internal representation of resources differs between the two tools. Internally, DataDog tends to merge

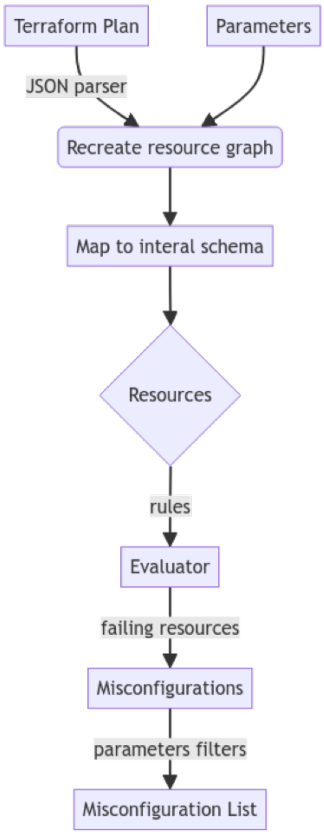

Figure 6.1: Implementation Architecture

multiple resource objects into a single schema, which simplifies detection queries by avoiding the need to examine multiple entities. However, this schema differs from the cloud provider specification, and it is not possible to directly map Terraform resources to it.

Creating a separate set of security checks that would only work with Infrastructure as Code resources is not an optimal solution, as it would require a significant engineering effort at scale. There are already hundreds of rules that operate on the company's internal schema.

As an example, the internal representation for an AWS S3 bucket has the following format (some details have been omitted for clarity).

```yaml
resource_type: aws_s3_bucket
labels:
  ...
display_name_fields:
  - bucket_name
external_identifier_field: bucket_arn
fields:
  # embedded non-resource in array
  - type: ARRAY_STRUCT
    name: access_block
    original_name: access_block
    nested_fields:
      - type: BOOLEAN
        name: block_public_acls
        original_name: BlockPublicAcls
        documentation: >
          Specifies whether ...
      - type: BOOLEAN
        ...
      ...
```

Snippet from the internal resource schema for an AWS S3 bucket

In this short snippet of the internal resource schema for an AWS S3 bucket, we can see that some fields have a `nested_field` property. This indicates that the value for the field will be added by the crawler from a separate API call, which in Terraform maps to a different resource type. Not only are different resources nested into a single one, but the field names often do not match those used by Terraform. To address this naming difference, we have defined configuration files that specify the field position in a specific Terraform resource and also indicate the corresponding field in the internal schema.

We have proposed a linear-time solution for merging child resources in order to address this issue.

```
def get_changed_aggregated_resources():
  nodes = {} # a set of resources
  changed_ref = {}
  res_plan = [resource_info from terraform] # planned
                                  resources
  for r in res_plan:
    if is_base_type(r): # for example aws_s3_bucket
      nodes.add(r)
      if r.has_changed:
        changed_ref.add(r)
  for r in res_plan:
    if not is_base_type(r): # for example aws_s3_bucket_acl
      nodes[r.bucket].subfields = r.values
      if r.has_changed:
        changed_ref.add(r.bucket)

  return nodes if node in changed_ref
```

Pseudo-code snippet of the resource aggregation logic

To implement the aforementioned algorithm, we need a list of mappings between resource types and the field that links to the base resource. In Terraform, this field varies for each resource type. For example, in bucket-related resources, the field that references the base resource is *bucket*, while it is different for other resource types. This implementation also allows us to identify which resources have changed, which is useful if the user only wants to run compliance checks on changed resources (e.g. in a pull/change request in version control).

6.3 OPA and rule structure

The Open Policy Agent (OPA) is used to evaluate security rules as shown in the central block of Figure 6.1. The OPA takes a set of resources and a set of rules to evaluate and determines whether the resources comply with the rules. OPA is designed to have a single policy language and policy model for all products and services in a system. Its policy expressions are declarative and use a high-level language that allows for easy iteration, hierarchy traversals, and string manipulations over JSON. OPA can be run as an HTTP service or imported as a library. In the CLI, both options were implemented. This allows users to run an OPA sidecar with a central set of rules loaded as the single source of truth for infrastructure configuration policy. The other option is to compile the rules into WebAssembly that can be loaded and evaluated on the client when executed.

Rule Structure

The rules evaluated by Open Policy Agent are written in Rego. The JSON object that is passed to the evaluator consists of several list fields that are named by the resource type that the plan contains.

```
{
    "aws_elasticace_cluster": [...],
    "aws_s3_bucket": [...],
    "aws_kms_key": [...]
}
```

Each list contains all the resources of the same type that can be found in the Terraform plan. This structure simplifies the creation of rules that apply only to specific resource types. This means a rule that will scan for miscofiguration of a specific resource type can be defined with the syntax in snippet 6.4.1.

```
expect_compliant(res) {
    res.values.field_we_want_to_check == "expected value"
}

findings[x] {
    res := input.resource_of_interest
    not expect_compliant(res)
    x := {
        "rule": "rule name",
        "compliant": false,
        "resource": res[_]
    }
}
```

Snippet 6.4.1. Example rule structure

When the rule in listing 6.4.1 is applied, an object containing a property called findings is returned. The object will have a compliance status and references to the misconfigured resources.

6.4 GDPR Rule Example

In the context of GDPR compliance, ensuring that sensitive data is properly logged and monitored is crucial for organizations operating in or with the European Union. The General Data Protection Regulation (GDPR) emphasizes the importance of data protection and privacy for individuals within the EU. As such, organizations must implement appropriate technical and organizational measures to ensure a level of security appropriate to the risk, including

the ability to ensure the ongoing confidentiality, integrity, availability, and resilience of processing systems and services.

Given this backdrop, let's consider a specific GDPR-related rule example that focuses on logging setups for AWS Elastic Load Balancers (ELBs). This is particularly relevant because ELBs can be used to distribute traffic to applications that handle personal data. Proper logging of these interactions is crucial for detecting, investigating, and recovering from security incidents, as well as for compliance with GDPR's accountability and data protection principles.

6.4.1 Example GDPR Rule for AWS ELB Logging

The rule ensures that AWS ELBs have logging enabled to capture and store logs of all requests made to the load balancer, thereby aiding in the detection and analysis of malicious traffic and potential data breaches. Below is a Rego policy snippet designed to check compliance with this requirement:

A logging setup should be created for AWS Elastic Load Balancers

```
# Expectation for compliant logging setup
expect_logging_enabled(res) {
    not is_null(res.access_logs)
    res.access_logs.bucket != ""
}

# Findings for AWS ELBs without proper logging setup
findings[resource_id] {
    resource := input.aws_elb[_]
    not expect_logging_enabled(resource)
    resource_id := {
        "rule": "AWS ELB Logging Not Enabled",
        "compliant": false,
        "resource_id": resource.id,
        "resource_type": "aws_elb"
    }
}
```

Snippet 6.4.1. Example GDPR rule

This Rego policy snippet defines a compliance check for AWS ELBs by ensuring the *access_logs* feature is configured, pointing to a valid logging bucket. The *expect_logging_enabled* function checks if the *access_logs* configuration exists and is not null, with a specified bucket where logs should be stored. If an ELB resource does not meet these criteria, it's flagged as non-compliant in the *findings* object, indicating a potential GDPR compliance issue.

6.4.2 Non-compliant terraform resource example

To illustrate a scenario where an AWS ELB is configured without the necessary logging, consider the following Terraform resource definition:

```
resource "aws_elb" "non_compliant_elb" {
  name               = "non-compliant-elb"
  availability_zones = ["us-west-2a", "us-west-2b", "us-
                                west-2c"]

  listener {
    instance_port     = 80
    instance_protocol = "HTTP"
    lb_port           = 80
    lb_protocol       = "HTTP"
  }

  health_check {
    healthy_threshold   = 2
    unhealthy_threshold = 2
    timeout             = 3
    target              = "HTTP:80/"
    interval            = 30
  }

  // Intentionally missing access_logs configuration to
                                simulate non-compliance
  // access_logs {
  //   bucket = "my-logging-bucket"
  //   interval = 60
  // }

  tags = {
    Name = "NonCompliantELB"
  }
}
```

This configuration snippet would be flagged by our Rego policy as non-compliant due to the absence of the *access_logs* configuration, highlighting a risk area for GDPR compliance.

6.5 Reporting non-compliant resources

The compliance tool includes a GitHub integration that allows it to be added as a step in an existing CI/CD pipeline. This allows users to scan for non-compliant resources on pull requests and commits. The tool is designed

to be used with the output of a Terraform plan command, which should be stored in a JSON file and passed as input to the program. The use of resource lists structured by type and the ability to create rules that apply to specific resource types simplifies the process of identifying misconfigurations and helps ensure the effectiveness of the scan.

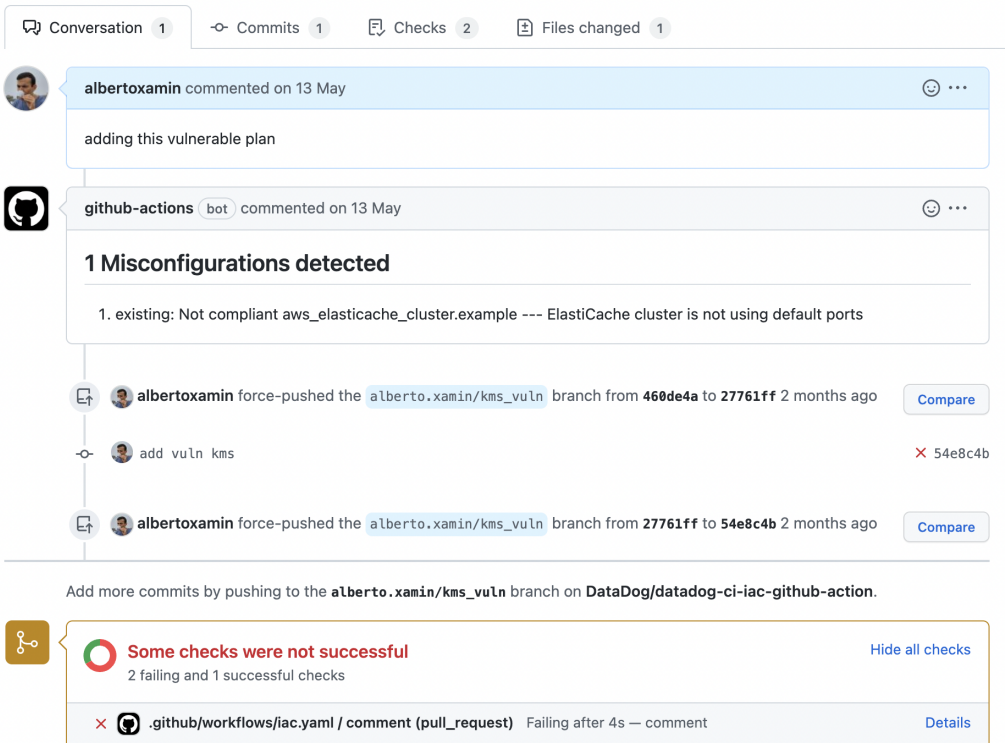

Figure 6.2: A comment on a Pull Request with the GitHub integration

Then the program outputs a list of "findings" that are used to generate comments or set a visible failure status to the changes, eventually making it impossible to merge and deploy a non compliant infrastructure.

6.6 Limitations

The current implementation of the compliance tool has some limitations that should be taken into account:

- Limited Resource Support: The tool currently supports a specific set of resource types from various cloud providers. While efforts have been made to cover a wide range of resources, there may still be resource

types that are not supported. Adding support for new resource types would require modifications to the tool.

- Static Evaluation: The compliance tool evaluates the configuration files statically by analyzing the Terraform plan. It does not consider runtime or dynamic factors that may affect the security posture of the resources. For example, it does not consider data from live environments or account-specific settings.

- Terraform Dependency Graph: The compliance tool relies on the Terraform dependency graph to identify the resources and their relationships. If there are issues with the dependency graph construction or if the graph is incomplete or inaccurate, it may affect the accuracy of the compliance evaluation.

- Rule Complexity: The rule structure in Rego used by the compliance tool may not be suitable for expressing complex rules or policies. Writing and maintaining complex rules may be challenging, especially for users who are not familiar with Rego or policy languages.

- Integration Limitations: The GitHub integration provided by the compliance tool allows for commenting on pull requests, but it may have limitations in terms of the depth of integration with CI/CD pipelines or other DevOps tools. Integrating the tool with other systems or workflows may require additional customization or development.

- Performance Considerations: The performance of the compliance tool may vary depending on the size and complexity of the Terraform plan and the number of rules being evaluated. Large plans or complex rules may increase the evaluation time, potentially impacting the CI/CD pipeline's overall execution time.

Chapter 7

Evaluation and discussion

In this thesis we successfully created a compliance tool for Cloud Posture Management that is able to read the outputs of terraform plans and report non-compliant resources before they would be deployed. The tool can be run locally or integrated with GitHub Actions a Continuous Integration (CI) platform.

The current implementation of the compliance tool has some limitations that need to be addressed in order to enhance its competitiveness in the market. One of the primary limitations is its restricted support for scanning only Terraform plan files. To make the tool more versatile, it is necessary to expand its capabilities to encompass a wider range of Infrastructure as Code tools and platforms. This can be achieved by further developing the internal schema mapping of resources and creating additional mapping files.

Another area of improvement lies in the location of the scanning logic. Currently, the scanning is limited to the local machine executing the scan, utilizing only the information available in the Infrastructure as Code file. Shifting the scanning logic to a cloud service within the company's infrastructure would enable evaluations that also consider infrastructure not managed by any Infrastructure as Code. By adopting this approach, a broader view of security issues can be attained, which might not be captured by the limited perspective of Infrastructure as Code tools.

Expanding the tool's support for additional compliance frameworks is another crucial aspect. Presently, the tool focuses on evaluating compliance against a specific set of rules. However, in the future, it could be enhanced to encompass multiple compliance frameworks, such as CIS Benchmarks or industry-specific standards. This extension would allow users to evaluate

their infrastructure against a broader range of compliance requirements, making the tool more adaptable to various industry standards.

To ensure ongoing compliance, introducing continuous monitoring capabilities would be beneficial. Rather than performing compliance evaluations as one-time scans, the tool can be extended to conduct regular scans of the infrastructure, detecting compliance violations in real-time. This would enable the tool to provide instant alerts or notifications when misconfigurations are identified, facilitating proactive remediation efforts.

In addition to identifying non-compliant resources, the compliance tool can offer remediation suggestions or best practices to assist users in addressing the detected misconfigurations. By providing actionable insights and guidance, users can effectively resolve compliance issues, making the tool more valuable in maintaining a compliant infrastructure.

Integrating the compliance tool with Security Incident and Event Management (SIEM) systems would provide centralized logging, monitoring, and reporting of compliance violations. This integration would offer a comprehensive view of compliance across the entire infrastructure, enabling improved incident response and security incident management.

Automation capabilities can be incorporated into the tool to automate the remediation of certain types of misconfigurations. This can be achieved through the use of Infrastructure as Code automation frameworks or direct interaction with cloud provider APIs to make configuration changes. Automated remediation would significantly streamline the compliance remediation process and reduce manual effort.

Empowering users to define their own custom rules through a custom rule engine would provide organizations with the flexibility to tailor compliance evaluations to their specific requirements and internal policies. This customization feature would enhance the tool's adaptability and make it more aligned with individual organizational needs.

Improving the reporting and visualization capabilities of the compliance tool would enhance user experience and facilitate better understanding and analysis of compliance results. This could involve generating detailed reports, visualizing compliance trends over time, and offering interactive dashboards for monitoring compliance status.

By integrating the compliance tool more tightly with Continuous Integration/Continuous Deployment (CI/CD) pipelines, automated compliance checks can be performed as part of the deployment process. This integration would enable the early detection of compliance issues in the development lifecycle, preventing the deployment of non-compliant configurations.

To implement these future works, extensive research, development, and collaboration with users and stakeholders would be required to prioritize and incorporate the desired features. Addressing these areas of improvement will enable the compliance tool to evolve and meet the evolving needs of users, providing more comprehensive and valuable compliance evaluation capabilities.

In conclusion, there are several directions for improving and expanding the compliance tool, including broadening its support for different Infrastructure as Code tools and platforms, incorporating comprehensive scanning logic, and enhancing usability

Chapter 8

Conclusion

This thesis aimed to develop a tool for integration into Datadog's product portfolio, with the primary objective of enhancing the efficiency in detecting non-compliant cloud infrastructure resources. The exploration began by tracing the transformation from traditional hardware-based infrastructure to cloud-based systems and the paradigm shift in managing infrastructure through version control systems, as detailed in Chapter 2. We then embarked on a journey to comprehend the intricacies of cloud compliance, exploring the multifaceted world of regulatory requirements and the associated challenges outlined in Chapter 2.

Our investigation extended to a survey of existing tools in the domain of cloud compliance and security, as documented in Chapter 3. However, as seen in the motivation chapter (Chapter 4), it became evident that Datadog's distinctive needs and the nuanced nature of cloud compliance necessitated the development of a bespoke tool tailored to address Datadog's specific requirements.

In summary, this research has encompassed a comprehensive journey through the evolution of cloud infrastructure, the significance of cloud compliance, and a survey of the state-of-the-art tools. The findings have laid the groundwork for the creation of an innovative solution poised to enhance Datadog's capability to maintain compliance in the ever-evolving cloud infrastructure landscape. As this academic pursuit concludes, we anticipate the forthcoming implementation and integration of this tool and its potential to streamline the cloud compliance process for Datadog and its customer base.

Bibliography

[1] Al-Safwani, N., Fazea, Y., and Ibrahim, H. Iscp: In-depth model for selecting critical security controls. *Computers & Security 77* (2018), 565–577.

[2] Ataya, G. Pci dss audit and compliance. *Information security technical report 15*, 4 (2010), 138–144.

[3] Brikman, Y. *Terraform: Up and Running.* " O'Reilly Media, Inc.", 2022.

[4] Burgess, M. A tiny overview of cfengine: Convergent maintenance agent. In *Proceedings of the 1st International Workshop on Multi-Agent and Robotic Systems, MARS/ICINCO* (2005), vol. 2005, Citeseer.

[5] De Kruijf, M. A., Sankaralingam, K., and Jha, S. Static analysis and compiler design for idempotent processing. In *Proceedings of the 33rd ACM SIGPLAN conference on Programming Language Design and Implementation* (2012), pp. 475–486.

[6] Deemer, P., Benefield, G., Larman, C., and Vodde, B. The scrum primer. *Scrum Primer is an in-depth introduction to the theory and practice of Scrum, albeit primarily from a software development perspective, available at: http://assets. scrumtraininginstitute. com/downloads/1/scrumprimer121. pdf 1285931497* (2010), 15.

[7] Fernandez, E. B., and Yimam, D. Towards compliant reference architectures by finding analogies and overlaps in compliance regulations. In *2015 12th International Joint Conference on e-Business and Telecommunications (ICETE)* (2015), vol. 04, pp. 435–440.

[8] Gikas, C. A general comparison of fisma, hipaa, iso 27000 and pci-dss standards. *Information Security Journal: A Global Perspective 19*, 3 (2010), 132–141.

[9] Guerriero, M., Garriga, M., Tamburri, D. A., and Palomba, F. Adoption, support, and challenges of infrastructure-as-code: Insights from industry. In *2019 IEEE international conference on software maintenance and evolution (ICSME)* (2019), IEEE, pp. 580–589.

[10] Gunawardena, J. *An introduction to idempotency.* Hewlett Packard, 1996.

[11] Hochstein, L., and Moser, R. *Ansible: Up and Running: Automating configuration management and deployment the easy way.* " O'Reilly Media, Inc.", 2017.

[12] Howard, M. Terraform–automating infrastructure as a service. *arXiv preprint arXiv:2205.10676* (2022).

[13] Ibrahim, A., Thiruvady, D., Schneider, J.-G., and Abdelrazek, M. The challenges of leveraging threat intelligence to stop data breaches. *Frontiers in Computer Science 2* (2020), 36.

[14] Kolokoltsov, V. N., and Maslov, V. P. *Idempotent analysis and its applications*, vol. 401. Springer Science & Business Media, 2013.

[15] Moore, W., and Frye, S. Review of hipaa, part 1: history, protected health information, and privacy and security rules. *Journal of nuclear medicine technology 47*, 4 (2019), 269–272.

[16] Morse, E. A., and Raval, V. Pci dss: Payment card industry data security standards in context. *Computer Law & Security Review 24*, 6 (2008), 540–554.

[17] Noor, U., Anwar, Z., Malik, A. W., Khan, S., and Saleem, S. A machine learning framework for investigating data breaches based on semantic analysis of adversary's attack patterns in threat intelligence repositories. *Future Generation Computer Systems 95* (2019), 467–487.

[18] Prassanna, J., Pawar, A., and Neelanarayanan, V. A review of existing cloud automation tools. *Asian J Pharm Clin Res 10* (2017), 471–473.

[19] Rhett, J. *Learning MCollective: Parallel Server Management in Puppet and Chef.* " O'Reilly Media, Inc.", 2014.

[20] Sabharwal, N., Pandey, S., Pandey, P., Sabharwal, N., Pandey, S., and Pandey, P. Getting started with hashicorp terraform. *Infrastructure-as-Code Automation Using Terraform, Packer,*

Vault, Nomad and Consul: Hands-on Deployment, Configuration, and Best Practices (2021), 11–45.

[21] Saleem, H., and Naveed, M. Sok: Anatomy of data breaches. *Proc. Priv. Enhancing Technol. 2020*, 4 (2020), 153–174.

[22] Schwaber, K. Scrum development process. In *Business Object Design and Implementation: OOPSLA'95 Workshop Proceedings 16 October 1995, Austin, Texas* (1997), Springer, pp. 117–134.

[23] Schwaber, K. *The enterprise and Scrum.* Microsoft press, 2007.

[24] Tankard, C. What the gdpr means for businesses. *Network Security 2016*, 6 (2016), 5–8.

[25] Yimam, D., and Fernandez, E. B. A survey of compliance issues in cloud computing. *Journal of Internet Services and Applications 7*, 1 (May 2016), 5.

[26] Zhou, M., Zhang, R., Zeng, D., and Qian, W. Services in the cloud computing era: A survey. In *2010 4th International Universal Communication Symposium* (2010), IEEE, pp. 40–46.

[27] Zolkifli, N. N., Ngah, A., and Deraman, A. Version control system: A review. *Procedia Computer Science 135* (2018), 408–415.